

AULA 2 · 16/06/2026

Framework Slim PHP

Construindo APIs REST profissionais com roteamento, controllers, models e middleware.

Slim 4

Versão do Framework

MVC

Padrão Arquitetural

4h

Duração da Aula

Por que usar um Framework?

Sem framework (PHP puro):

- Roteamento manual com `$_SERVER[ 'REQUEST_URI' ]`
- Headers CORS repetidos em cada arquivo
- Sem organização padronizada de código
- Difícil de escalar e manter
- Código duplicado em vários lugares

Funciona para projetos pequenos, mas vira um caos rapidamente.

Com Slim Framework:

- **Roteamento** elegante e declarativo
- **Middleware** para CORS, autenticação, logs
- **PSR-7** — objetos Request e Response padronizados
- **Estrutura MVC** organizada
- **Testável** e extensível

Slim é um *micro-framework* — fornece o essencial sem impor muita estrutura.
Ideal para APIs!

Composer — Gerenciador de Dependências

O Composer é para PHP o que o **npm** é para Node.js. Gerencia bibliotecas e o autoloading de classes.

INSTALAR O SLIM (TERMINAL)

```
# Criar projeto
composer init

# Instalar Slim 4 e PSR-7
composer require slim/slim
composer require slim/psr7

# Ou criar um projeto direto:
composer create-project slim/slim-skeleton meu-projeto
```

COMPOSER.JSON — AUTOLOAD PSR-4

```
{
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "require": {
    "slim/slim": "^4.0",
    "slim/psr7": "^1.0"
  }
}
```

Após instalar, execute:

```
# Gera o autoloader
composer dump-autoload
```

Estrutura criada pelo Composer:

```
meu-projeto/
├─ vendor/           # dependências
│  └─ autoload.php
│     └─ slim/
├─ src/              # nosso código
├─ public/
│  └─ index.php      # ponto de entrada
└─ composer.json
```

Nunca versione a pasta vendor/ no Git. Use .gitignore.

Estrutura do Projeto APISlim

Estrutura MVC do nosso projeto:

```
APISlim/
├─ public/
│   └─ index.php      🔑 Ponto de entrada
├─ src/
│   ├─ Controllers/
│   │   └─ ServicoController.php
│   ├─ Models/
│   │   ├─ Database.php    Singleton PDO
│   │   └─ Servico.php     Model
│   ├─ Middlewares/
│   │   └─ CorsMiddleware.php
│   └─ Utils/
│       └─ Response.php    JSON padronizado
├─ config/
│   └─ config.php         Configurações
├─ vendor/                Composer
└─ composer.json
```

Responsabilidade de cada camada:

Camada	Responsabilidade
index.php	App Slim, rotas, bootstrap
Controller	Recebe Request, chama Model, retorna Response
Model	Acessa banco de dados, lógica de negócio
Middleware	Processa antes/depois das rotas
Utils	Helpers reutilizáveis
Config	Parâmetros de ambiente

Toda requisição passa por: **index.php** → **Middleware** → **Controller** → **Model** → **Controller** → **Response**

public/index.php — O Coração da API

PUBLIC/INDEX.PHP

```
<?php
use Slim\Factory\AppFactory;
use App\Controllers\ServicoController;
use App\Middlewares\CorsMiddleware;

require dirname(__DIR__) . '/vendor/autoload.php';
require dirname(__DIR__) . '/config/config.php';

// 1. Criar a aplicação Slim
$app = AppFactory::create();

// 2. Definir base path (para Laragon)
$app->setBasePath('/eng_prog3_2026_1/APISlim/public');

// 3. Adicionar Middlewares
$app->addBodyParsingMiddleware(); // parseia JSON do body
$app->addRoutingMiddleware();
$app->add(new CorsMiddleware());

// 4. Definir rotas
$app->get('/api/health', function($req, $res) {
    return $res->withJson(['status' => 'online']);
});

$app->get('/api/servicos', [ServicoController::class, 'index']);
$app->get('/api/servicos/{id}', [ServicoController::class, 'show']);
```

Roteamento no Slim 4

O Slim mapeia **métodos HTTP** + **URL** para funções ou métodos de controllers:

Tipos de rotas:

GET	/api/servicos	Listar todos
GET	/api/servicos/{id}	Buscar um
POST	/api/servicos	Criar
PUT	/api/servicos/{id}	Atualizar
DELETE	/api/servicos/{id}	Deletar

PARÂMETROS DE ROTA

```
// {id} captura o valor da URL
$app->get('/api/servicos/{id}',
function($request, $response, $args) {
    $id = (int) $args['id']; // URL: /api/servicos/5
    // $id === 5
    return $response;
});
```

QUERY STRING E BODY

```
// Query string: GET /api/servicos?status=ativo
$params = $request->getQueryParams();
$status = $params['status'] ?? null;

// Body JSON: POST /api/servicos
$body = $request->getParsedBody();
$titulo = $body['titulo'] ?? null;

// Headers da requisição
$token = $request->getHeaderLine('Authorization');

// Agrupar rotas com prefixo
$app->group('/api', function($group) {
    $group->get('/servicos', '...');
    $group->get('/clientes', '...');
});
```

PSR-7 — Request & Response

O PSR-7 é um padrão que define **interfaces imutáveis** para mensagens HTTP. Todo controller recebe e retorna objetos PSR-7.

OBJETO REQUEST

```
// Assinatura de todo handler
function(Request $request, Response $response, array $args): Response

// Lendo dados do Request
$request->getMethod();           // "GET", "POST"...
$request->getUri()->getPath();     // "/api/servicos"
$request->getParsedBody();        // array do JSON body
$request->getQueryParams();       // array da query string
$request->getHeaderLine('Accept'); // valor do header
```

OBJETO RESPONSE

```
// Response é IMUTÁVEL – cada método retorna NOVO objeto!

// Escrever JSON manualmente
$payload = json_encode(['ok' => true]);
$response->getBody()->write($payload);
return $response->withHeader(
    'Content-Type', 'application/json'
)->withStatus(200);

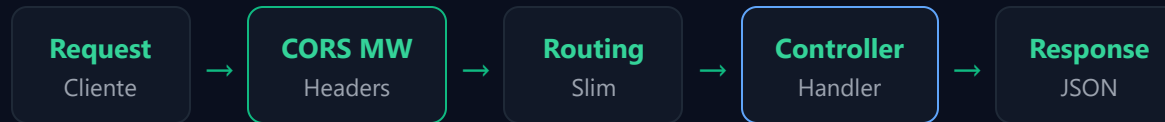
// Definir código de status
return $response->withStatus(404);
return $response->withStatus(201);

// Adicionar header
return $response->withHeader('X-Custom', 'valor');
```

O nosso `Response.php` (utilitário) encapsula a lógica de resposta JSON para não repetir esse código em todos os controllers.

Middlewares

Middleware é código que executa **antes e/ou depois** do handler da rota. Ideal para CORS, autenticação, logs e validação.



SRC/MIDDLEWARES/CORSMIDDLEWARE.PHP

```
<?php
namespace App\Middlewares;

use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\RequestHandlerInterface as Handler;

class CorsMiddleware {
    public function __invoke(Request $request, Handler $handler) {
        // Trata OPTIONS (preflight do browser)
        if ($request->getMethod() === 'OPTIONS') {
            $response = new \Slim\Psr7\Response();
        } else {
            $response = $handler->handle($request); // executa o handler
        }

        // Adiciona headers CORS à resposta
        return $response
            ->withHeader('Access-Control-Allow-Origin', '*')
            ->withHeader('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS')
```

ServicoController — Camada de Controle

SRC/CONTROLLERS/SERVICOCONTROLLER.PHP

```
<?php
namespace App\Controllers;

use App\Models\Servico;
use App\Utils\Response;
use Psr\Http\Message\ServerRequestInterface as Req;
use Psr\Http\Message\ResponseInterface as Res;

class ServicoController {

    /** GET /api/servicos */
    public function index(Req $req, Res $res): Res {
        $servicos = (new Servico())->getAll();
        return Response::json($res, $servicos);
    }

    /** GET /api/servicos/{id} */
    public function show(Req $req, Res $res, array $args): Res {
        $servico = (new Servico())->find((int) $args['id']);
        if (!$servico) {
            return Response::error($res, 'Não encontrado', 404);
        }
        return Response::json($res, $servico);
    }

    /** POST /api/servicos */
```

O controller deve ser "magro":

- Recebe a requisição
- Valida os dados de entrada
- Chama o Model
- Retorna a resposta

Não deve:

- Conter queries SQL diretamente
- Fazer lógica de negócio complexa
- Acessar banco sem usar o Model

Separação de responsabilidades torna o código testável e fácil de manter.

Os métodos index, show, store, update, destroy seguem a convenção REST dos frameworks.

Servico Model — Camada de Dados

SRC/MODELS/SERVICO.PHP

```
<?php
namespace App\Models;

class Servico {
    private $db;

    public function __construct() {
        $this->db = Database::getInstance()->getConnection();
    }

    public function getAll(): array {
        $stmt = $this->db->query("SELECT * FROM servicos ORDER BY id DESC");
        return $stmt->fetchAll();
    }

    public function find(int $id): ?array {
        $stmt = $this->db->prepare("SELECT * FROM servicos WHERE id = :id");
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ? : null;
    }

    public function create(array $data): int {
        $sql = "INSERT INTO servicos (titulo, descricao, preco)
                VALUES (:titulo, :descricao, :preco)";
        $stmt = $this->db->prepare($sql);
        $stmt->execute($data);
        return $this->db->lastInsertId();
    }
}
```

SRC/MODELS/DATABASE.PHP — SINGLETON

```
<?php
namespace App\Models;

class Database {
    private static ?Database $instance = null;
    private PDO $connection;

    private function __construct() {
        $config = require dirname(__DIR__, 2) . '/config/config.php';
        $dsn = "mysql:host={$config['db']['host']};dbname={$config['db']['name']}";
        $this->connection = new PDO($dsn,
            $config['db']['user'],
            $config['db']['pass'],
            [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
             PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
        );
    }

    public static function getInstance(): self {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function getConnection(): PDO {
        return $this->connection;
    }
}
```

Utils/Response.php — Respostas Padronizadas

SRC/UTILS/RESPONSE.PHP

```
<?php
namespace App\Utils;

use Psr\Http\Message\ResponseInterface;

class Response {

    /** Resposta de sucesso */
    public static function json(
        ResponseInterface $res,
        $data,
        int $status = 200
    ): ResponseInterface {
        $payload = json_encode([
            'success' => true,
            'data'    => $data,
        ], JSON_UNESCAPED_UNICODE);
        $res->getBody()->write($payload);
        return $res
            ->withHeader('Content-Type', 'application/json')
            ->withStatus($status);
    }

    /** Resposta de erro */
    public static function error(
        ResponseInterface $res,
        string $message,
        int $status = 404
    ): ResponseInterface {
        $payload = json_encode([
            'success' => false,
            'error'    => $message,
        ], JSON_UNESCAPED_UNICODE);
        $res->getBody()->write($payload);
        return $res->withStatus($status);
    }
}
```

Exemplos de resposta gerados:

```
// Sucesso - GET /api/servicos
{
  "success": true,
  "data": [
    { "id": 1, "titulo": "Hidráulica" },
    { "id": 2, "titulo": "Elétrica" }
  ]
}

// Criado - POST /api/servicos (HTTP 201)
{
  "success": true,
  "data": { "id": 3 }
}

// Erro - GET /api/servicos/99 (HTTP 404)
{
  "success": false,
  "error": "Não encontrado"
}
```

Consistência nas respostas facilita muito o desenvolvimento do app React Native.

Configurações da API

CONFIG/CONFIG.PHP

```
<?php
return [
    // Banco de dados
    'db' => [
        'host' => 'localhost',
        'name' => 'trabalho',
        'user' => 'root',
        'pass' => '',
    ],

    // CORS - origens permitidas
    'cors' => [
        'allowed_origins' => [
            'http://localhost:8081', // React Native
            'http://localhost:3000', // React Web
            'http://localhost:19000', // Expo
        ],
    ],

    // Ambiente
    'app' => [
        'env' => 'development',
        'debug' => true,
        'name' => 'APISlim',
    ],
];
```

Boas práticas de configuração:

- **Nunca** colocar senhas no controle de versão
- Em produção, use variáveis de ambiente (.env)
- Centralizar toda configuração em um arquivo

USANDO VARIÁVEIS DE AMBIENTE (.ENV)

```
# .env (não commitar!)
DB_HOST=localhost
DB_NAME=trabalho
DB_USER=root
DB_PASS=senha_segura

# PHP - Ler com getenv()
return [
    'db' => [
        'host' => getenv('DB_HOST') ?: 'localhost',
        'name' => getenv('DB_NAME') ?: 'trabalho',
        'user' => getenv('DB_USER') ?: 'root',
        'pass' => getenv('DB_PASS') ?: '',
    ],
];
```

CRUD Completo — Fluxo de uma Requisição

Exemplo: **PUT /api/servicos/3** — atualizar serviço

CONTROLLER — UPDATE()

```
public function update(Req $req, Res $res, array $args): Res {
    $id    = (int) $args['id'];
    $data  = (array) $req->getParsedBody();
    $model = new Servico();

    // 1. Verificar se existe
    if (!$model->find($id)) {
        return Response::error($res, 'Não encontrado', 404);
    }

    // 2. Validar
    if (empty($data['titulo'])) {
        return Response::error($res, 'Título obrigatório', 400);
    }

    // 3. Atualizar
    $model->update($id, $data);

    // 4. Retornar registro atualizado
    return Response::json($res, $model->find($id));
}
```

MODEL — UPDATE() E DESTROY()

```
public function update(int $id, array $data): bool {
    $sql = "UPDATE servicos
            SET titulo = :titulo,
              descricao = :descricao,
              preco = :preco
            WHERE id = :id";
    $stmt = $this->db->prepare($sql);
    return $stmt->execute([
        ':titulo'    => trim($data['titulo']),
        ':descricao' => $data['descricao'] ?? '',
        ':preco'     => (float) ($data['preco'] ?? 0),
        ':id'        => $id,
    ]);
}

public function destroy(int $id): bool {
    $stmt = $this->db->prepare(
        "DELETE FROM servicos WHERE id = :id"
    );
    return $stmt->execute([':id' => $id]);
}
```

Testando a API

cURL (terminal):

```
# Listar serviços
curl http://localhost/eng_prog3_2026_1/APISlim/public/api/servicos

# Criar serviço
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Pintura","preco":300}' \
  http://localhost/.../api/servicos

# Atualizar serviço
curl -X PUT \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Pintura Premium","preco":400}' \
  http://localhost/.../api/servicos/1

# Deletar serviço
curl -X DELETE http://localhost/.../api/servicos/1

# Verificar saúde da API
curl http://localhost/.../api/health
```

Postman / Insomnia (GUI):



Postman

Interface gráfica para testar APIs. Permite salvar coleções de requisições e compartilhar com a equipe.



Insomnia

Alternativa mais leve ao Postman. Ótimo para desenvolvimento rápido.

Veja o arquivo APISlim/TESTES_CURL.md no projeto para os exemplos completos de teste.

.htaccess — Roteamento no Apache

Para que o Slim receba **todas as requisições** e faça o roteamento, o Apache precisa redirecionar tudo para o `index.php`:

PUBLIC/.HTACCESS

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

O que isso faz:

1. Ativa o módulo de rewrite do Apache
2. Se a URL não é um arquivo real (!-f)
3. E não é uma pasta real (!-d)
4. Redireciona para `index.php`

Sem o .htaccess, URLs como `/api/servicos` retornariam 404 do Apache. O Slim nunca seria chamado.

Verifique se o módulo `mod_rewrite` está ativo no Apache do Laragon: **Laragon**
→ **Apache** → **httpd.conf**

Integração com React Native

No app React Native, usamos a `api/api.js` para centralizar as chamadas:

```
// ServicosApp/api/api.js
const BASE_URL = 'http://192.168.x.x/eng_prog3_2026_1/APISlim/public/api';
// 🍷 IP local da máquina onde o Laragon roda

export const getServicos = async () => {
  const res = await fetch(`${BASE_URL}/servicos`);
  const json = await res.json();
  return json.data; // array de serviços
};

export const criarServico = async (dados) => {
  const res = await fetch(`${BASE_URL}/servicos`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(dados),
  });
  return res.json();
};
```

Uso no componente React Native:

```
// screens/ListaServicos.js
import { getServicos } from '../api/api';

const [servicos, setServicos] = useState([]);

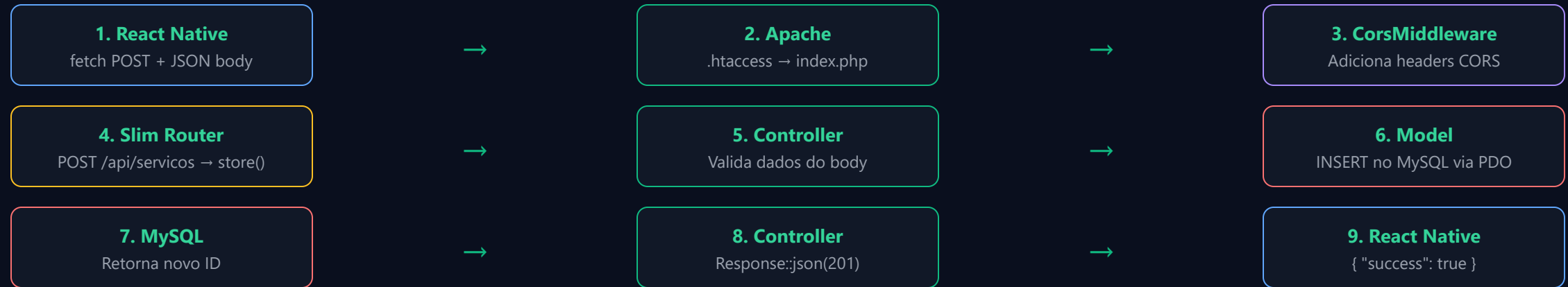
useEffect(async () => {
  const data = await getServicos();
  setServicos(data);
}, []);
```

⚠ **Importante:** No emulador Android, use `10.0.2.2` em vez de `localhost`. No dispositivo físico, use o IP da sua rede local.

O CORS Middleware que criamos permite que o React Native consuma a API sem erros de segurança.

Fluxo Completo MVC

Visualizando o caminho de uma requisição **POST /api/servicos**:



Erros Comuns e Soluções

Erro	Causa	Solução
404 Not Found	Base path incorreto	Verificar <code>\$app->setBasePath()</code> e <code>.htaccess</code>
CORS Error	Headers CORS ausentes	Adicionar <code>CorsMiddleware</code> e tratar <code>OPTIONS</code>
500 Error	Erro PHP/BD	Ver log Apache: <code>C:\laragon\logs\apache\</code>
Body vazio	Faltou <code>addBodyParsingMiddleware</code>	Adicionar antes das rotas no <code>index.php</code>
Autoload falha	Namespace errado	Verificar <code>composer.json</code> e executar <code>composer dump-autoload</code>
JSON inválido	BOM ou espaço antes de <code><?php</code>	Remover espaços/linhas antes da tag PHP

Dica de debug: Adicione `error_reporting(E_ALL);` e `ini_set('display_errors', 1);` no início do `index.php` durante o desenvolvimento.

Exercício Prático — Aula 2

Com o projeto APISlim/ já configurado, implemente:

1. Execute o `setup.sql` no phpMyAdmin para criar a tabela `servicos`.
2. Acesse `http://localhost/eng_prog3_2026_1/APISlim/public/api/health` e verifique se retorna `{"status":"online"}`.
3. Use o cURL ou Postman para criar 3 serviços via `POST /api/servicos`.
4. Liste todos os serviços com `GET /api/servicos`.
5. Atualize um serviço com `PUT /api/servicos/{id}`.
6. **Desafio:** Adicione um endpoint `GET /api/servicos/buscar?titulo=xyz` para busca por texto.
7. **Desafio 2:** Adicione validação para que o preço nunca seja negativo.

Consulte `APISlim/TESTES_CURL.md` para os exemplos de requisição completos e `APISlim/GUIA_INTEGRACAO.md` para o guia detalhado da aula.

Expansões e Próximos Passos



Autenticação JWT

Adicionar tokens JWT para proteger endpoints.
Middleware de autenticação.



Paginação

GET /api/servicos?
page=1&limit=10 —
retornar dados paginados.



Busca e Filtros

GET /api/servicos?
titulo=xyz&preco_max=500



Upload de Arquivos

Fotos dos serviços. Slim
suporta upload via
getUploadedFiles().



Logs

Monolog para registro de
erros e acesso em arquivos
de log.



Deploy

Publicar em servidor real:
Hostinger, Railway,
DigitalOcean.

Veja APISlim/EXPANSOES_E_DESAFIOS.md para ideias completas de extensão do projeto.

PHP Puro vs Slim — Comparativo

Aspecto	PHP Puro	Slim Framework
Roteamento	Manual com switch/if	Declarativo e limpo
CORS	Header em todo arquivo	Middleware uma vez
Organização	Livre (vira bagunça)	MVC estruturado
PSR-7	Não	Sim — padrão da indústria
Testabilidade	Difícil	Fácil (injeção de dependência)
Curva de aprendizado	Menor	Um pouco maior
Escalabilidade	Limitada	Alta
Tamanho (sem vendor)	~5 KB	~100 KB (Slim core)

Conclusão: Para projetos didáticos e profissionais, o Slim vale o investimento. O PHP puro é ótimo para entender os fundamentos, mas o Slim nos prepara para o mercado.

Resumo — O que aprendemos



Estrutura MVC

Controllers, Models, Middlewares e Utils organizados em src/.



Roteamento Slim

get, post, put, delete com parâmetros de rota.



PSR-7

Request e Response imutáveis — padrão da indústria PHP.



Middleware

CORS e BodyParsing para processar antes/depois das rotas.



Singleton Database

Uma única conexão PDO compartilhada em toda a aplicação.



Integração React Native

API consumida com fetch. CORS configurado para app mobile.

Próxima aula (23/06): React Native — consumir a API, formulários de cadastro, listagem e exclusão de serviços!